In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

In [6]:
```
df= pd.read_csv(r'C:\Users\Lenovo\Downloads\housing.csv\housing.csv')
```

In [7]:
```
df.shape
```

Out[7]:
```
(20640, 10)
```

In [8]:
```
df.head
```

Out[8]:
```
<bound method NDFrame.head of       longitude  latitude  housing_median_age  tot
al_rooms  total_bedrooms  \
0       -122.23     37.88                41.0        880.0           129.0
1       -122.22     37.86                21.0       7099.0          1106.0
2       -122.24     37.85                52.0       1467.0           190.0
3       -122.25     37.85                52.0       1274.0           235.0
4       -122.25     37.85                52.0       1627.0           280.0
...         ...       ...                 ...          ...             ...
20635   -121.09     39.48                25.0       1665.0           374.0
20636   -121.21     39.49                18.0        697.0           150.0
20637   -121.22     39.43                17.0       2254.0           485.0
20638   -121.32     39.43                18.0       1860.0           409.0
20639   -121.24     39.37                16.0       2785.0           616.0

       population  households  median_income  median_house_value  \
0           322.0       126.0         8.3252            452600.0
1          2401.0      1138.0         8.3014            358500.0
2           496.0       177.0         7.2574            352100.0
3           558.0       219.0         5.6431            341300.0
4           565.0       259.0         3.8462            342200.0
...           ...         ...            ...                 ...
20635       845.0       330.0         1.5603             78100.0
20636       356.0       114.0         2.5568             77100.0
20637      1007.0       433.0         1.7000             92300.0
20638       741.0       349.0         1.8672             84700.0
20639      1387.0       530.0         2.3886             89400.0

      ocean_proximity
0            NEAR BAY
1            NEAR BAY
2            NEAR BAY
3            NEAR BAY
4            NEAR BAY
...               ...
20635          INLAND
20636          INLAND
20637          INLAND
20638          INLAND
20639          INLAND

[20640 rows x 10 columns]>
```

In [9]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  str    
dtypes: float64(9), str(1)
memory usage: 1.7 MB
```

In [10]:
```
df.describe
```

Out[10]:
```
<bound method NDFrame.describe of       longitude  latitude  housing_median_age
total_rooms  total_bedrooms  \
0        -122.23     37.88                41.0        880.0           129.0
1        -122.22     37.86                21.0       7099.0          1106.0
2        -122.24     37.85                52.0       1467.0           190.0
3        -122.25     37.85                52.0       1274.0           235.0
4        -122.25     37.85                52.0       1627.0           280.0
...          ...       ...                 ...          ...             ...
20635    -121.09     39.48                25.0       1665.0           374.0
20636    -121.21     39.49                18.0        697.0           150.0
20637    -121.22     39.43                17.0       2254.0           485.0
20638    -121.32     39.43                18.0       1860.0           409.0
20639    -121.24     39.37                16.0       2785.0           616.0

       population  households  median_income  median_house_value  \
0           322.0       126.0         8.3252            452600.0
1          2401.0      1138.0         8.3014            358500.0
2           496.0       177.0         7.2574            352100.0
3           558.0       219.0         5.6431            341300.0
4           565.0       259.0         3.8462            342200.0
...           ...         ...            ...                 ...
20635       845.0       330.0         1.5603             78100.0
20636       356.0       114.0         2.5568             77100.0
20637      1007.0       433.0         1.7000             92300.0
20638       741.0       349.0         1.8672             84700.0
20639      1387.0       530.0         2.3886             89400.0

      ocean_proximity
0            NEAR BAY
1            NEAR BAY
2            NEAR BAY
3            NEAR BAY
4            NEAR BAY
...               ...
20635          INLAND
20636          INLAND
20637          INLAND
20638          INLAND
20639          INLAND

[20640 rows x 10 columns]>
```

In [11]:
```
df.columns
```

Out[11]:
```
Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
       'total_bedrooms', 'population', 'households', 'median_income',
       'median_house_value', 'ocean_proximity'],
      dtype='str')
```

In [12]:
```
df.isnull().sum()
```

Out[12]:
```
longitude              0
latitude               0
housing_median_age     0
total_rooms            0
total_bedrooms       207
population             0
households             0
median_income          0
median_house_value     0
ocean_proximity        0
dtype: int64
```

In [13]:
```
df.duplicated().sum()
```

Out[13]:
```
np.int64(0)
```

In [14]:
```
df["total_bedrooms"] = df["total_bedrooms"].fillna(df["total_bedrooms"].median())
```

In [16]:
```
df.isnull().sum()
```

Out[16]:
```
longitude             0
latitude              0
housing_median_age    0
total_rooms           0
total_bedrooms        0
population            0
households            0
median_income         0
median_house_value    0
ocean_proximity       0
dtype: int64
```

In [17]:
```
df.columns
```

Out[17]:
```
Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
       'total_bedrooms', 'population', 'households', 'median_income',
       'median_house_value', 'ocean_proximity'],
      dtype='str')
```

In [21]:
```
x = df.drop("median_house_value",axis=1)
y = df["median_house_value"]
```

In [22]:
```
x_train, x_test,y_train ,y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42)
```

In [23]:
```
print("x_train:",x_train.shape)
print("x_test:",x_test.shape)
print("y_train:",y_train.shape)
print("y_test:",y_test.shape)
```

```
x_train: (16512, 9)
x_test: (4128, 9)
y_train: (16512,)
y_test: (4128,)
```

In [26]:
```
x = pd.get_dummies(
    x,
    columns=["ocean_proximity"],
    drop_first=True
)
```

In [28]:
```
x.columns
```

Out[28]:
```
Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
       'total_bedrooms', 'population', 'households', 'median_income',
       'ocean_proximity_INLAND', 'ocean_proximity_ISLAND',
       'ocean_proximity_NEAR BAY', 'ocean_proximity_NEAR OCEAN'],
      dtype='str')
```

In [29]:
```
x = x.astype(float)
```

In [30]:
```
x.dtypes
```

Out[30]:
```
longitude                   float64
latitude                    float64
housing_median_age          float64
total_rooms                 float64
total_bedrooms              float64
population                  float64
households                  float64
median_income               float64
ocean_proximity_INLAND      float64
ocean_proximity_ISLAND      float64
ocean_proximity_NEAR BAY    float64
ocean_proximity_NEAR OCEAN  float64
dtype: object
```

In [32]:
```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42
)
```

In [33]:
```
scaler = StandardScaler()

x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

In [36]:
```
lr = LinearRegression()
lr.fit(x_train_scaled, y_train)
y_pred_lr = lr.predict(x_test_scaled)
```

In [37]:
```
mae_lr = mean_absolute_error(y_test,y_pred_lr)
mse_lr = mean_squared_error(y_test,y_pred_lr)
rmse_lr = np.sqrt(mse_lr)
r2_lr = r2_score(y_test, y_pred_lr)

print("Linear Regression")
print("MAE:",mae_lr)
print("MSE:",mse_lr)
```

```
print("RMSE:",rmse_lr)
print("R2:",r2_lr)
```

```
Linear Regression
MAE: 50670.73824097189
MSE: 4908476721.156615
RMSE: 70060.52184473518
R2: 0.6254240620553606
```

In [38]:
```
#Ridge Regression
ridge = Ridge(alpha=1.0)
ridge.fit(x_train_scaled,y_train)
y_pred_ridge=ridge.predict(x_test_scaled)
```

In [39]:
```
mae_ridge = mean_absolute_error(y_test, y_pred_ridge)
mse_ridge = mean_squared_error(y_test,y_pred_ridge)
rmse_ridge = np.sqrt(mse_ridge)
r2_ridge = r2_score(y_test,y_pred_ridge)
print("Ridge Regression")
print("MAE:",mae_ridge)
print("MSE:",mse_ridge)
print("RMSE:",rmse_ridge)
print("R2:",r2_ridge)
```

```
Ridge Regression
MAE: 50668.12293144744
MSE: 4908041658.44779
RMSE: 70057.41686964907
R2: 0.6254572625840543
```

In [40]:
```
#Lasso Regression
lasso = Lasso(alpha=0.01)
lasso.fit(x_train_scaled,y_train)
y_pred_lasso = lasso.predict(x_test_scaled)
```

In [41]:
```
mae_lasso = mean_absolute_error(y_test, y_pred_lasso)
mse_lasso = mean_squared_error(y_test,y_pred_lasso)
rmse_lasso = np.sqrt(mse_lasso)
r2_lasso = r2_score(y_test, y_pred_lasso)
print("Lasso Regression")
print("MAE: ",mae_lasso)
print("MSE: ",mse_lasso)
print("RMSE: ",rmse_lasso)
print("R2: ",r2_lasso)
```

```
Lasso Regression
MAE:  50670.73218289936
MSE:  4908475779.947158
RMSE:  70060.51512761776
R2:  0.6254241338809872
```

In [42]:
```
#Elastic Net
elastic = ElasticNet(
    alpha=0.01,
    l1_ratio=0.5
)
elastic.fit(x_train_scaled,y_train)
y_pred_elastic=elastic.predict(x_test_scaled)
```

In [46]:
```
mae_elastic = mean_absolute_error(y_test,y_pred_elastic)
mse_elastic = mean_squared_error(y_test,y_pred_elastic)
rmse_elastic = np.sqrt(mse_elastic)
r2_elastic = r2_score(y_test, y_pred_elastic)
print("Elastic Net")
print("MAE: ",mae_elastic)
print("MSE: ",mse_elastic)
print("RMSE: ",rmse_elastic)
print("r2: ", r2_elastic)
```

```
Elastic Net
MAE:  50527.983936881414
MSE:  4886913207.188927
RMSE:  69906.46041095864
r2:  0.6270696180860157
```

In [47]:
```
results = pd.DataFrame({
    "Model": [
        "Linear Regression",
        "Ridge",
        "Lasso",
        "Elastic Net"
    ],
    "MAE": [
        mae_lr,
        mae_ridge,
        mae_lasso,
        mae_elastic
    ],
    "MSE": [
        mse_lr,
        mse_ridge,
        mse_lasso,
        mse_elastic
    ],
    "RMSE": [
        rmse_lr,
        rmse_ridge,
        rmse_lasso,
        rmse_elastic
    ],
    "R2": [
        r2_lr,
        r2_ridge,
        r2_lasso,
        r2_elastic
    ]
})

results
```

Out[47]:

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	50670.738241	4.908477e+09	70060.521845	0.625424
1	Ridge	50668.122931	4.908042e+09	70057.416870	0.625457
2	Lasso	50670.732183	4.908476e+09	70060.515128	0.625424
3	Elastic Net	50527.983937	4.886913e+09	69906.460411	0.627070

In [49]:

```
#Compare coefficients
coefficients = pd.DataFrame({
    "Feature": x.columns,
    "Linear": lr.coef_,
    "Ridge": ridge.coef_,
    "Lasso": lasso.coef_,
    "ElasticNet": elastic.coef_
})

coefficients
```

Out[49]:

	Feature	Linear	Ridge	Lasso	ElasticNet
0	longitude	-53826.648016	-53713.651593	-53826.359145	-46067.391630
1	latitude	-54415.696144	-54298.400415	-54415.406029	-46356.268731
2	housing_median_age	13889.866189	13890.472329	13889.859513	13916.771413
3	total_rooms	-13094.251162	-13058.643741	-13094.040844	-10539.872594
4	total_bedrooms	43068.181842	42978.328403	43067.954114	37321.886168
5	population	-43403.432427	-43389.968779	-43403.359260	-42155.261665
6	households	18382.196324	18425.412550	18382.149926	20510.815853
7	median_income	75167.774766	75157.635752	75167.725268	74397.564774
8	ocean_proximity_INLAND	-18506.095225	-18538.725530	-18506.181176	-20759.134541
9	ocean_proximity_ISLAND	2118.437572	2119.249166	2118.429656	2170.109228
10	ocean_proximity_NEAR BAY	-1621.705018	-1616.222261	-1621.672205	-1218.996377
11	ocean_proximity_NEAR OCEAN	1139.905592	1146.564026	1139.916142	1607.359402

In [50]:

```
coefficients.sort_values(
    by="Lasso",
    key=abs,
    ascending=False
)
```

Out[50]:

	Feature	Linear	Ridge	Lasso	ElasticNet
7	median_income	75167.774766	75157.635752	75167.725268	74397.564774
1	latitude	-54415.696144	-54298.400415	-54415.406029	-46356.268731
0	longitude	-53826.648016	-53713.651593	-53826.359145	-46067.391630
5	population	-43403.432427	-43389.968779	-43403.359260	-42155.261665
4	total_bedrooms	43068.181842	42978.328403	43067.954114	37321.886168
8	ocean_proximity_INLAND	-18506.095225	-18538.725530	-18506.181176	-20759.134541
6	households	18382.196324	18425.412550	18382.149926	20510.815853
2	housing_median_age	13889.866189	13890.472329	13889.859513	13916.771413
3	total_rooms	-13094.251162	-13058.643741	-13094.040844	-10539.872594
9	ocean_proximity_ISLAND	2118.437572	2119.249166	2118.429656	2170.109228
10	ocean_proximity_NEAR BAY	-1621.705018	-1616.222261	-1621.672205	-1218.996377
11	ocean_proximity_NEAR OCEAN	1139.905592	1146.564026	1139.916142	1607.359402

In [51]:
```
coefficients[
    coefficients["Lasso"] == 0
]
```

Out[51]:

Feature	Linear	Ridge	Lasso	ElasticNet

In [53]:
```
alphas = [0.001, 0.01, 0.1, 1, 10, 100, 1000]

ridge_results = []

for alpha in alphas:

    model = Ridge(alpha=alpha)
    model.fit(x_train_scaled, y_train)

    prediction = model.predict(x_test_scaled)

    rmse = np.sqrt(mean_squared_error(y_test, prediction))
    r2 = r2_score(y_test, prediction)

    ridge_results.append([alpha, rmse, r2])

ridge_results = pd.DataFrame(
    ridge_results,
    columns=["Alpha", "RMSE", "R2"]
)

ridge_results
```

Out[53]:

	Alpha	RMSE	R2
0	0.001	70060.518718	0.625424
1	0.010	70060.490584	0.625424
2	0.100	70060.209426	0.625427
3	1.000	70057.416870	0.625457
4	10.000	70031.305311	0.625736
5	100.000	69891.656726	0.627228
6	1000.000	70378.379797	0.622018

In [54]:

```
alphas = [0.0001, 0.001, 0.01, 0.1, 1, 10]

lasso_results = []

for alpha in alphas:

    model = Lasso(alpha=alpha, max_iter=10000)
    model.fit(x_train_scaled, y_train)

    prediction = model.predict(x_test_scaled)

    rmse = np.sqrt(mean_squared_error(y_test, prediction))
    r2 = r2_score(y_test, prediction)

    zero_coefficients = np.sum(model.coef_ == 0)

    lasso_results.append([
        alpha,
        rmse,
        r2,
        zero_coefficients
    ])

lasso_results = pd.DataFrame(
    lasso_results,
    columns=[
        "Alpha",
        "RMSE",
        "R2",
        "Zero_Coefficients"
    ]
)

lasso_results
```

Out[54]:

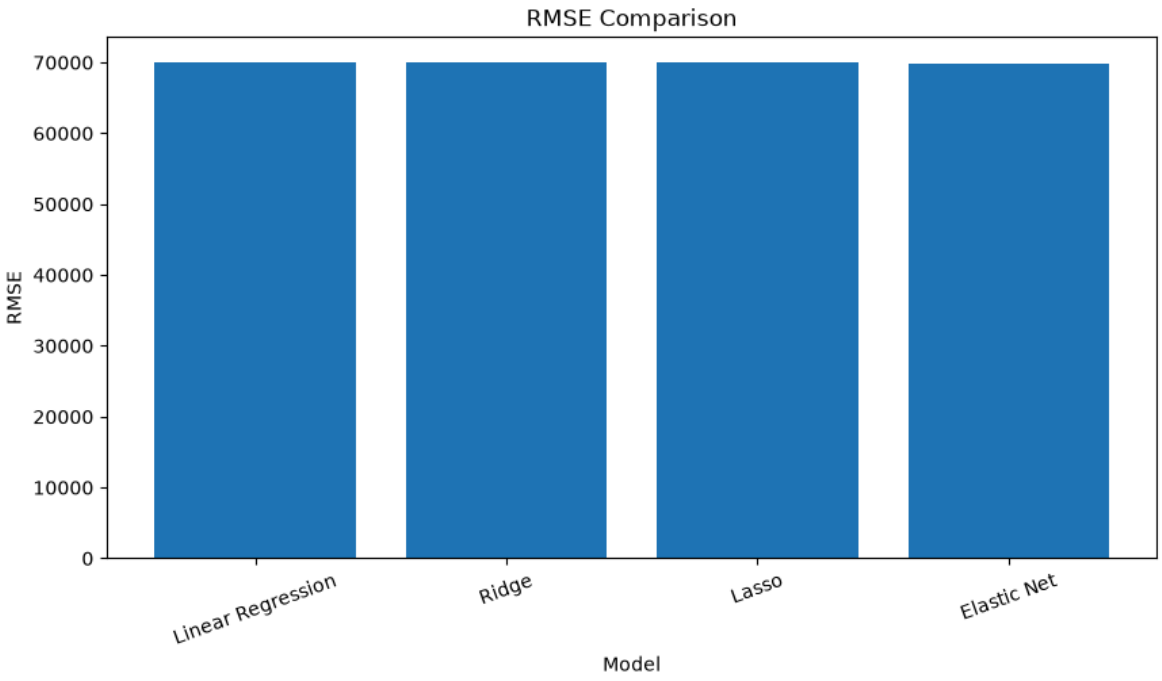
	Alpha	RMSE	R2	Zero_Coefficients
0	0.0001	70060.521778	0.625424	0
1	0.0010	70060.521172	0.625424	0
2	0.0100	70060.515128	0.625424	0
3	0.1000	70060.454594	0.625425	0
4	1.0000	70059.851178	0.625431	0
5	10.0000	70053.904216	0.625495	0

In [55]:
```
plt.figure(figsize=(10, 5))

plt.bar(
    results["Model"],
    results["RMSE"]
)

plt.xlabel("Model")
plt.ylabel("RMSE")
plt.title("RMSE Comparison")

plt.xticks(rotation=20)
plt.show()
```



In [56]:
```
final_model = Ridge(alpha=10)

final_model.fit(x_train_scaled, y_train)

final_predictions = final_model.predict(x_test_scaled)
```

In [57]:
```
print("Final Model Performance")
print("-----------------------")

print(
```

```
    "MAE:",
    mean_absolute_error(y_test, final_predictions)
)

print(
    "RMSE:",
    np.sqrt(
        mean_squared_error(y_test, final_predictions)
    )
)

print(
    "R²:",
    r2_score(y_test, final_predictions)
)
```

```
Final Model Performance
-----------------------
MAE: 50646.17065906168
RMSE: 70031.30531058174
R²: 0.6257364071110391
```

In [ ]: